# Reporte del Proyecto

## Tienda de Comida Rápida

Aplicación web desarrollada en Java con Spring Boot, Thymeleaf y MySQL

**Integrantes:**

Fabian Amir Coronado Trejo

Carlos Alberto Estrada Jimenez

**Materia:** Programación Orientada a Objetos

**Fecha:** 26/10/2026

---

## 1. Introducción

Este reporte describe, de forma resumida, el proyecto de una tienda de comida rápida desarrollado en IntelliJ IDEA con Java. El sistema permite administrar usuarios, productos, clientes, empleados y quejas, guardando toda la información en una base de datos MySQL.

El documento explica paso a paso la arquitectura del proyecto, las clases (modelo, repositorios y configuración), los controladores y el procedimiento para crear la base de datos con seis tablas: usuario, contraseña, productos, clientes, empleados y quejas.

## 2. Tecnologías y configuración del proyecto

El proyecto usa Gradle como herramienta de construcción. Los datos principales del archivo build.gradle son:

| Elemento | Valor |
|---|---|
| Nombre del proyecto | clinica67 |
| Grupo / paquete base | com.fct |
| Versión | 0.0.1-SNAPSHOT |
| Lenguaje | Java (toolchain versión 27) |
| Framework | Spring Boot 4.1.1 |
| Base de datos | MySQL (local, puerto 3306, esquema tienda) |

| Elemento | Valor |
|---|---|
| Clase principal | com.fct.clinica67.Clinica67Application |

Las dependencias declaradas y su función son:

- **spring-boot-starter-webmvc:** permite crear controladores web y manejar peticiones HTTP (patrón MVC).
- **spring-boot-starter-thymeleaf:** motor de plantillas para generar las páginas HTML dinámicas.
- **spring-boot-starter-data-jpa:** acceso a la base de datos mediante entidades y repositorios, sin escribir SQL manual.
- **spring-boot-starter-validation:** validación de los formularios (campos obligatorios, longitudes, formatos).
- **spring-boot-starter-security:** inicio de sesión y protección de las páginas; las contraseñas se guardan cifradas.
- **mysql-connector-j:** controlador JDBC para conectar Java con MySQL.
- **Dependencias de prueba:** JUnit y los starters de test para probar el proyecto.

## 3. Estructura del proyecto

La estructura sigue el patrón MVC (Modelo, Vista, Controlador) con una capa adicional de repositorios:

```
src/main/java/com/fct/clinica67/
 ├── Clinica67Application.java   (clase principal)
 ├── model/        (entidades: Usuario, Contrasena, Producto, Cliente, Empleado,
Queja)
 ├── repository/   (interfaces de acceso a datos)
 ├── controller/   (controladores web)
 └── config/       (seguridad)
src/main/resources/
 ├── application.properties   (conexión a la base de datos)
 ├── templates/               (páginas HTML de Thymeleaf)
 └── static/                  (CSS, imágenes)
```

*Nota: los nombres de las clases de las secciones 4 y 5 corresponden a la estructura estándar para las seis tablas del sistema; deben ajustarse si en el código real tienen otro nombre.*

## 4. Clases del proyecto (paso a paso)

### Paso 1. Clase principal: Clinica67Application

Es el punto de entrada del sistema. Está anotada con @SpringBootApplication y contiene el método main, que arranca el servidor de Spring Boot. Al ejecutarse, Spring busca todas las clases del paquete, configura la conexión con MySQL a partir del archivo application.properties y deja la aplicación disponible en el navegador. Cuando se presiona el botón Run en IntelliJ, lo que se ejecuta es esta clase.

### Paso 2. Clases del modelo (entidades)

Las entidades son clases Java que representan las tablas de la base de datos tienda. Cada una se marca con @Entity y @Table, y cada atributo equivale a una columna. El identificador lleva @Id y @GeneratedValue para que MySQL asigne el número de forma automática. Sobre los atributos se

colocan validaciones como @NotBlank, @Size o @Email, para evitar que se guarden datos vacíos o incorrectos. Además, cada clase incluye su constructor, sus getters y sus setters.

| Clase | Tabla | Qué representa |
|---|---|---|
| Usuario | usuario | Persona con acceso al sistema (nombre, correo, rol). |
| Contrasena | contrasena | Contraseña cifrada de cada usuario (relación uno a uno con Usuario). |
| Producto | productos | Alimentos y bebidas del menú (nombre, descripción, categoría, precio, disponibilidad). |
| Cliente | clientes | Personas que compran en la tienda (nombre, teléfono, correo, dirección). |
| Empleado | empleados | Personal de la tienda (nombre, puesto, teléfono, salario, fecha de contratación). |
| Queja | quejas | Reclamos de los clientes (asunto, descripción, estado, fecha). |

Las relaciones entre las clases se expresan con anotaciones: Usuario y Contrasena se enlazan con @OneToOne, y Cliente y Queja con @OneToMany y @ManyToOne, de modo que cada queja sabe a qué cliente pertenece.

### Paso 3. Repositorios

Los repositorios son interfaces que extienden JpaRepository. No hace falta escribir SQL ni su implementación, porque Spring genera automáticamente los métodos para guardar, buscar, actualizar y eliminar registros. Existe un repositorio por cada entidad: UsuarioRepository, ContrasenaRepository, ProductoRepository, ClienteRepository, EmpleadoRepository y QuejaRepository. El de usuarios agrega además el método findByCorreo, necesario para el inicio de sesión.

### Paso 4. Configuración de seguridad

Como el proyecto incluye Spring Security, todas las páginas piden inicio de sesión por defecto. La clase SecurityConfig define qué rutas son públicas (inicio, registro y archivos CSS) y cuáles requieren sesión o rol de administrador. También declara el cifrador de contraseñas BCrypt, que transforma la contraseña antes de guardarla en la tabla contrasena, y un servicio que consulta al usuario en la base de datos para validar su acceso.

### Paso 5. Vistas (Thymeleaf)

Son archivos HTML ubicados en src/main/resources/templates. Reciben los datos del controlador y los muestran con etiquetas como th:each, para recorrer listas, y th:object, para los formularios. Hay una vista de lista y una de formulario por cada módulo: productos, clientes, empleados y quejas.

## 5. Controladores

Los controladores están en el paquete controller y se anotan con @Controller. Reciben la petición del navegador, usan los repositorios para consultar o guardar datos, y devuelven el nombre de la vista HTML. Todos los módulos siguen el mismo esquema CRUD:

| Ruta | Método | Acción |
|---|---|---|
| /modulo | GET | Muestra la lista de registros |
| /modulo/nuevo | GET | Muestra el formulario vacío |
| /modulo/guardar | POST | Valida y guarda (con @Valid y BindingResult) |
| /modulo/editar/{id} | GET | Carga un registro en el formulario |
| /modulo/eliminar/{id} | GET | Elimina el registro |

Controladores del sistema:

**HomeController** (/): muestra la página de inicio con el menú de la tienda.

**UsuarioController** (/login, /registro, /usuarios): permite registrar usuarios; al hacerlo guarda el usuario y su contraseña cifrada en la tabla contrasena.

**ProductoController** (/productos): administra el menú de productos, con su categoría, precio y disponibilidad.

**ClienteController** (/clientes): registra y consulta a los clientes de la tienda.

**QuejaController** (/quejas): el cliente registra una queja y el administrador cambia su estado (Pendiente, En revisión o Resuelta).

**EmpleadoController** (/empleados): alta, consulta, edición y baja del personal.

## 6. Base de datos en MySQL (paso a paso)

El proyecto se conecta a una base de datos local llamada tienda, en localhost:3306, con el usuario root. En esta sección se explica cómo crearla desde cero y cómo enlazarla con la aplicación. Los comandos SQL pueden ejecutarse desde la consola de IntelliJ o desde cualquier cliente de MySQL.

### Paso 1. Iniciar el servidor

Abrir el servidor MySQL o MariaDB (por ejemplo, desde el panel de XAMPP) y comprobar que esté activo en el puerto 3306.

### Paso 2. Conectarse

En IntelliJ se abre la pestaña Database, se presiona el botón +, luego Data Source y MySQL. Se indica el host localhost, el puerto 3306 y el usuario root, y se prueba la conexión con Test Connection. Si el proyecto ya tenía una fuente de datos configurada, solo hay que cambiar el nombre de la base a tienda. También es posible conectarse desde la consola con el comando mysql -u root -p.

### Paso 3. Crear la base de datos

Primero se crea la base tienda con codificación utf8mb4, que permite guardar acentos y la letra ñ, y después se selecciona con USE.

```
CREATE DATABASE IF NOT EXISTS tienda
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_general_ci;

USE tienda;
```

### Paso 4. Crear las seis tablas

Las tablas se crean en el orden que se muestra, porque algunas dependen de otras mediante llaves foráneas.

```
CREATE TABLE usuario (
  id_usuario     INT AUTO_INCREMENT PRIMARY KEY,
  nombre         VARCHAR(100) NOT NULL,
  correo         VARCHAR(120) NOT NULL UNIQUE,
  rol            ENUM('CLIENTE','EMPLEADO','ADMIN') NOT NULL DEFAULT 'CLIENTE',
  fecha_registro TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE contrasena (
  id_contrasena       INT AUTO_INCREMENT PRIMARY KEY,
  id_usuario          INT NOT NULL UNIQUE,
  contrasena_hash     VARCHAR(255) NOT NULL,
  fecha_actualizacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
    ON DELETE CASCADE
);

CREATE TABLE productos (
  id_producto INT AUTO_INCREMENT PRIMARY KEY,
  nombre      VARCHAR(100) NOT NULL,
  descripcion VARCHAR(255),
  categoria   VARCHAR(50) NOT NULL,
  precio      DECIMAL(10,2) NOT NULL CHECK (precio > 0),
  disponible  BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE clientes (
  id_cliente INT AUTO_INCREMENT PRIMARY KEY,
  nombre     VARCHAR(100) NOT NULL,
  telefono   VARCHAR(15),
  correo     VARCHAR(120),
  direccion  VARCHAR(200)
);

CREATE TABLE empleados (
  id_empleado        INT AUTO_INCREMENT PRIMARY KEY,
  id_usuario         INT NULL,
  nombre             VARCHAR(100) NOT NULL,
  puesto             VARCHAR(60) NOT NULL,
```

```
  telefono           VARCHAR(15),
  salario            DECIMAL(10,2) NOT NULL,
  fecha_contratacion DATE NOT NULL,
  FOREIGN KEY (id_usuario) REFERENCES usuario(id_usuario)
    ON DELETE SET NULL
);

CREATE TABLE quejas (
  id_queja    INT AUTO_INCREMENT PRIMARY KEY,
  id_cliente  INT NOT NULL,
  id_producto INT NULL,
  asunto      VARCHAR(120) NOT NULL,
  descripcion TEXT NOT NULL,
  estado      ENUM('PENDIENTE','EN_REVISION','RESUELTA') NOT NULL DEFAULT
'PENDIENTE',
  fecha       TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (id_cliente)  REFERENCES clientes(id_cliente),
  FOREIGN KEY (id_producto) REFERENCES productos(id_producto)
    ON DELETE SET NULL
);
```

La tabla usuario guarda a las personas con acceso al sistema y su rol. La tabla contrasena guarda por separado el hash BCrypt de cada usuario, nunca la contraseña en texto plano, y se escribe sin ñ para evitar problemas de codificación. La tabla productos contiene el menú con su categoría, precio y disponibilidad. En clientes se registran los datos de contacto de quienes compran, en empleados el personal de la tienda con su puesto y salario, y en quejas los reclamos de los clientes con su estado: pendiente, en revisión o resuelta.

### Paso 5. Relaciones entre las tablas

Cada usuario tiene una sola contraseña cifrada (usuario y contrasena, relación uno a uno). Un cliente puede registrar varias quejas (clientes y quejas, uno a muchos), y una queja puede referirse a un producto, aunque este dato es opcional. Un empleado también puede tener una cuenta de usuario para entrar al sistema, de forma opcional.

### Paso 6. Verificar la creación

Para comprobar que todo quedó bien se ejecuta SHOW TABLES;, que debe mostrar las seis tablas: usuario, contrasena, productos, clientes, empleados y quejas. Si se desea, también se pueden agregar registros de prueba con la instrucción INSERT INTO.

### Paso 7. Conectar el proyecto con la base de datos

En el archivo src/main/resources/application.properties se escribe la conexión; la contraseña depende de la instalación de cada equipo.

```
spring.datasource.url=jdbc:mysql://localhost:3306/tienda
spring.datasource.username=root
spring.datasource.password=TU_CONTRASEÑA
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
```

Con ddl-auto en validate, Hibernate comprueba que las entidades coincidan con las tablas creadas. Si se prefiere que las tablas se generen automáticamente a partir de las clases, se puede usar update.

## 7. Cómo ejecutar el proyecto

1. Abrir la carpeta del proyecto en IntelliJ IDEA y esperar a que Gradle descargue las dependencias.
2. Tener MySQL en ejecución con la base tienda creada.
3. Ejecutar la clase Clinica67Application (botón Run) o el comando ./gradlew bootRun (en Windows, gradlew.bat bootRun).
4. Abrir el navegador en http://localhost:8080.

Archivos de apoyo del repositorio: gradlew y gradlew.bat (ejecutan Gradle sin instalarlo), .gitignore (evita subir archivos temporales como build y .idea) y .gitattributes (normaliza los saltos de línea).

## 8. Conclusiones

El proyecto aplica la programación orientada a objetos y el patrón MVC: las entidades representan las tablas, los repositorios acceden a los datos, los controladores gestionan las peticiones y las vistas Thymeleaf muestran la información. La base de datos de seis tablas permite administrar usuarios, contraseñas cifradas, productos, clientes, quejas y empleados de la tienda de comida rápida, y Spring Security protege el acceso al sistema.

## 9. Código fuente

Carpeta de Google Drive con los archivos del código del proyecto:

https://drive.google.com/drive/folders/1EKnM7CGfOsfnCFGjBKRnyr4qOwywZYI1?usp=sharing